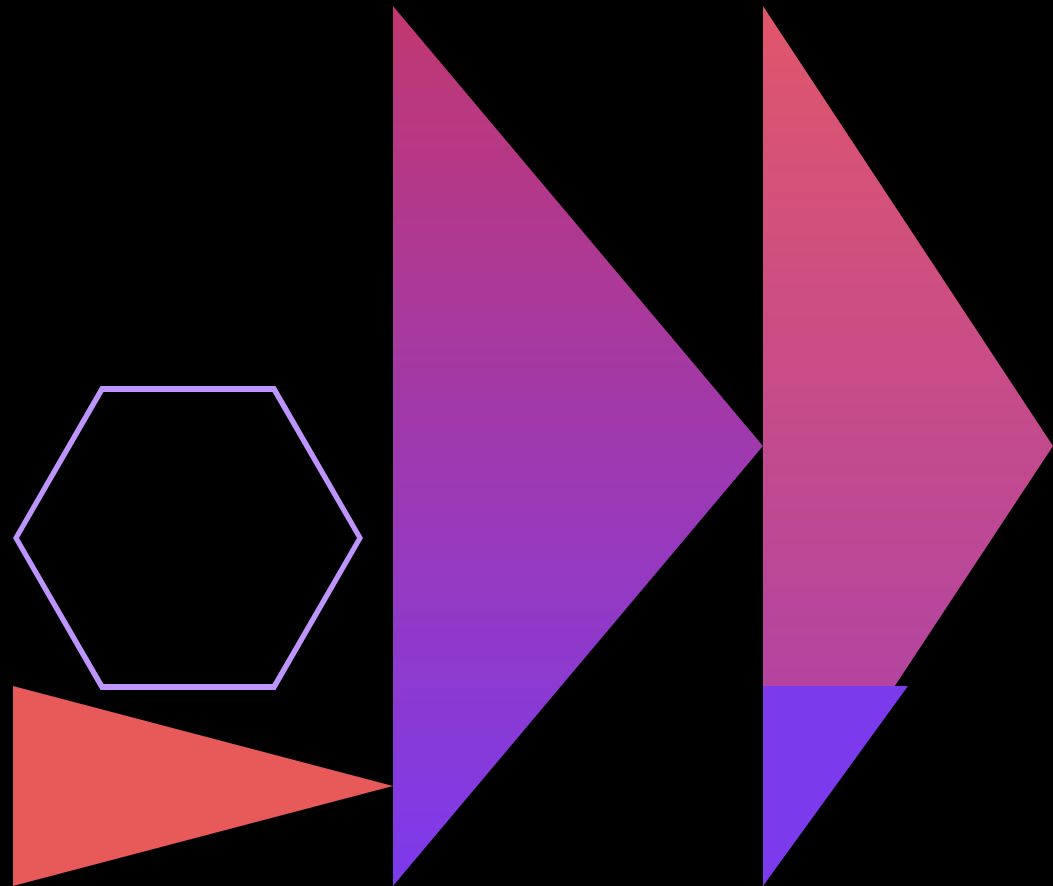
Solving MLOps Complexity: How Bob Accelerates OpenShift AI Workflows

LAB-1512 · IBM TechXchange 2026

Aisha Mohammed Farooq Darga
AI Engineer, Service Engineering - US Select Territory

Martin Wu
Partner Technical Specialist - IBM Service Partners



IBM **TechXchange**

Instructor-led Lab Guide Team

AF

Aisha Mohammed Farooq Darga
AI Engineer, Service Engineering - US Select Territory

–
aisdarg1@in.ibm.com

MW

Martin Wu
Partner Technical Specialist - IBM Service Partners

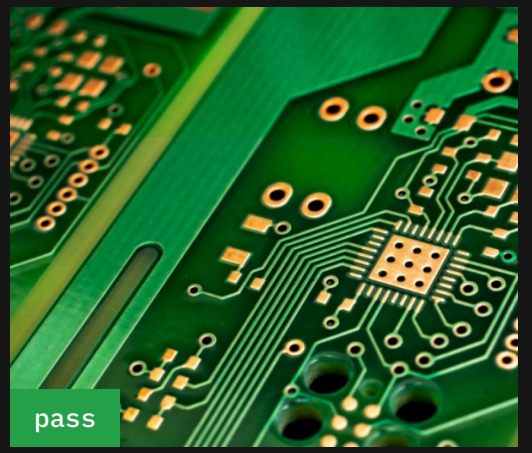
–
Ziyan.Wu@ibm.com

THE PROBLEM

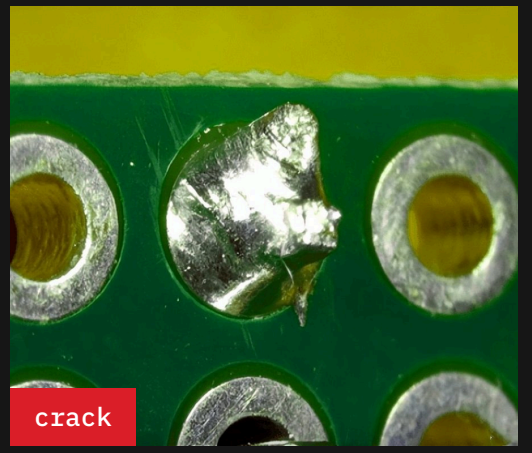
MLOps is **too complex** to move fast

Getting a model to train is hard enough. Getting it into production on OpenShift — YAML, routes, runtimes, namespaces — is where most teams slow down or stop entirely.

today's task	PCB defect inspection: catch scratches, cracks & contamination before products ship
the hard part	Going from a trained model file to a live REST endpoint on Kubernetes
the answer	IBM Bob removes the friction — you describe the goal, Bob handles the platform





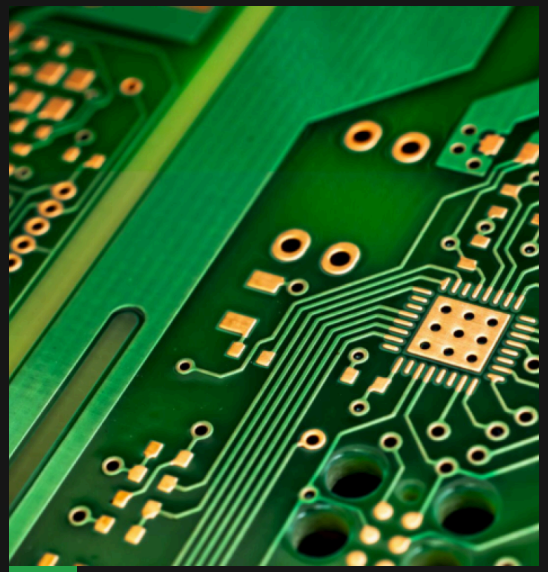




THE USE CASE

Printed circuit board defect classification

Four classes. One model. Catch defects before the product ships.



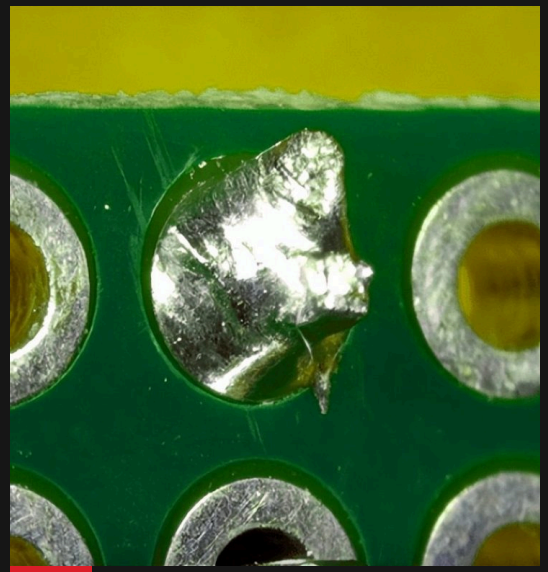
Pass

Clean board — no defects detected. Clears inspection.



Scratch

Surface abrasion damages signal traces once in the field.



Crack

Structural fracture causes intermittent failures → warranty claims.



Contamination

Foreign particles risk short-circuits in the field → product recall.

WHAT BOB HELPS YOU BUILD

From raw images to a deployed app — accelerated by Bob

The complete MLOps lifecycle on OpenShift AI. At every step, Bob handles the platform so you focus on the AI.

01

Explore data

Bob explains the dataset structure and helps you understand what the model needs to learn.

`JupyterLab · PIL`

02

Fine-tune YOLOv8

Bob generates the training config and monitors the run — you stay focused on the goal.

`YOLOv8 · PyTorch`

03

Export ONNX

Bob walks you through the export and validates the output is serving-ready.

`ONNX · NumPy`

04

Serve with KServe

Bob writes the ServingRuntime and InferenceService YAML and applies it for you.

`KServe · OpenVINO`

05

Inspection app

Bob deploys the full stack — FastAPI, PostgreSQL, nginx — from a single prompt.

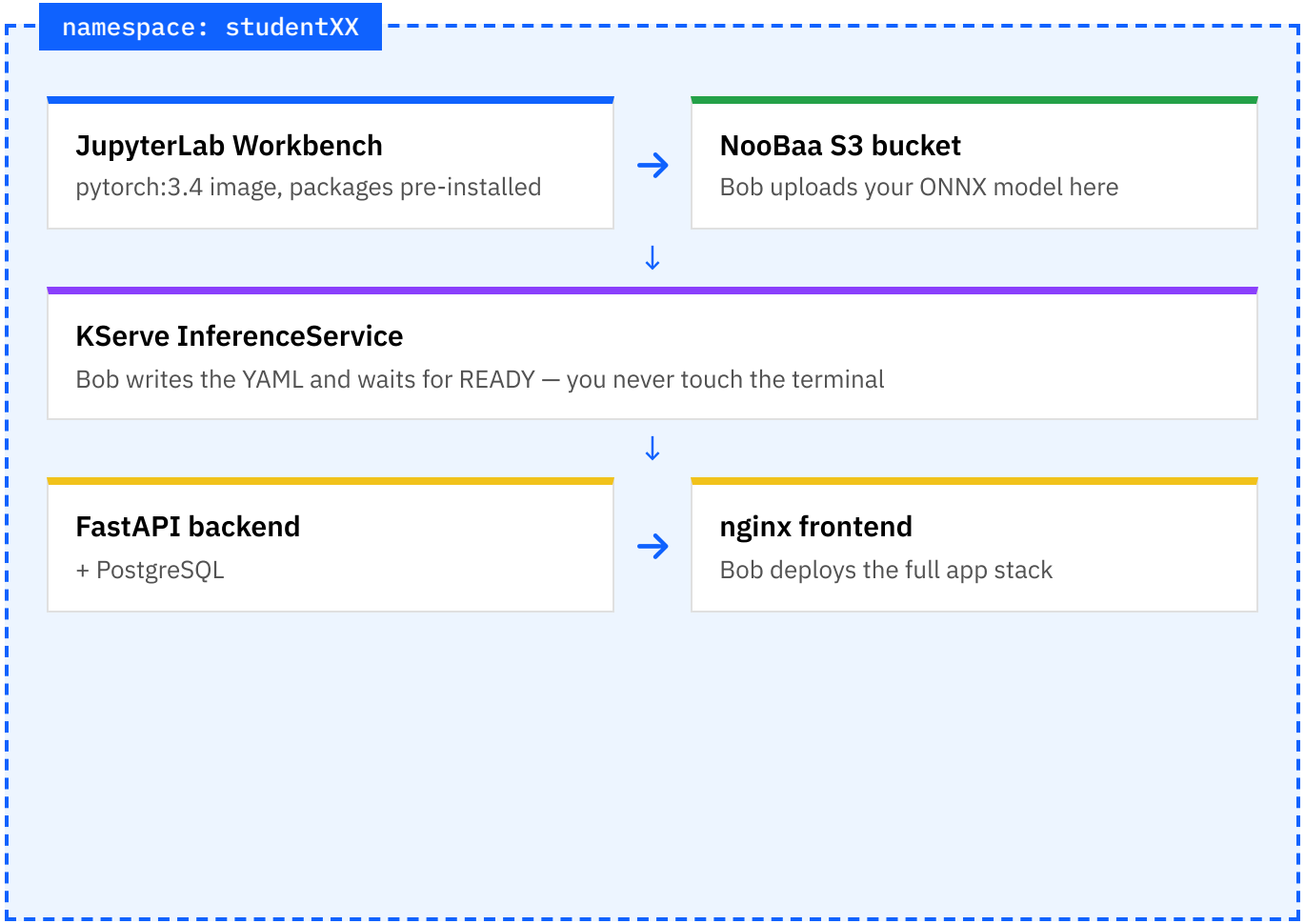
`FastAPI · PostgreSQL · nginx`

Bob turns every platform step into a conversation. Same result, fraction of the time.

LAB ARCHITECTURE

Your isolated namespace — Bob connects every layer

Everything runs in `studentXX` on the cluster. Bob knows the stack and talks to all of it.



Bob handles YAML

ServingRuntime, InferenceService, Routes — Bob writes and applies them from a single prompt.

Bob knows OpenShift

Namespace boundaries, headless services, KServe quirks — Bob has the context baked in.

Nothing on your laptop

Every tool — ultralytics, onnxruntime, boto3, torch — runs in the cluster.

IBM BOB — SOLVING MLOPS COMPLEXITY

Plain English in. Running OpenShift AI service out.

The platform work that used to block you — YAML, routes, runtimes, namespace config — becomes a conversation with Bob.

YOU

Deploy my ONNX model as a KServe inference service.

IBM BOB

```
Creating ServingRuntime… applying InferenceService…
route ready ✓
```

YOU

Deploy the inspection app with FastAPI and PostgreSQL.

IBM BOB

```
Applying deployments… services… app is live at your
route ✓
```

YOUR ROADMAP TODAY

01	**Validate** your environment	02	**Explore** image classification
03	**Fine-tune** and export the model	04	**Deploy** the model with KServe
05	**Test** inference from Python	06	**Deploy** the full-stack app
07	**End-to-end** test in the browser	08	**Stretch:** batch classification

Start here: set your Student ID → `studentXX`